Claude Code in Practice

Basics · Demo · Scenario 1 · Scenario 2

Agenda

- 🧱 Basics — Building blocks of Claude Code
- 🎬 Demo — Simpsons Agent live
- 🔬 Scenario 1 — Building a service from scratch
- 🔄 Scenario 2 — Spring Boot → Quarkus migration

Basics

The building blocks of Claude Code

Basics

📄 agent.md

Defines the agent's identity, goals, behavior, and operating instructions

🔔 Hooks

Run custom logic before or after specific Claude actions to automate checks and side effects

🔑 Permissions

Controls what the agent is allowed to access or execute — files, shell, network

Basics (cont.)

🤖 Agents

Modular, reusable AI units with specific roles — compose complex workflows from focused components

🛠️ Skills

Packaged abilities the agent can invoke for specialized tasks beyond code

⚡ Slash Commands

Shortcut commands that invoke predefined actions via simple /text inputs

MCP

Model Context Protocol — connecting agents to external tools, data, and services

📦 The Harness

Packages all building blocks into one deployable unit

agent.md · Hooks · Permissions · Agents · Skills · Slash Commands · MCP

Demo

Simpsons Agent

Scenario 1

Building a Laboratory Observation Service with Quarkus

Scenario 1 — Description

- 🏗️ Quarkus-based Laboratory Observation Service
- 🤖 AI model: Claude Sonnet
- 📄 Specs-first, phased approach

Scenario 1 — Approach

1 Pick Model → **2** Create /specs → **3** Add Requirements → **4** PLAN.md First → **5** Iterate

Scenario 1 — Approach Details

- Model: Claude Sonnet
- Folder /specs with a zipped example service
- Brief technical requirements
- Functional requirements
- Phased: ask for PLAN.md before coding

Scenario 1 — Result

✅ What Worked

- First running vertical slice in ~20 mins
- Rapid code generation
- Clean initial structure

⚠️ What Didn't

- Omitted 'extensions' in phase 1
- Model chose Jakarta Repositories over PanacheRepository
- Tests failed due to StatelessSession limitations

💡 Takeaway — Harness Engineering

Added explicit directive: always use PanacheRepository

→ Refining your specs and instructions for future runs is called "Feedback flywheel"

🔧 Small prompt adjustments → big reliability gains

Scenario 2

Spring Boot → Quarkus Migration

Scenario 2 — Description

- 🔄 Convert Spring Boot services to Quarkus
- 💻 Two services: person-service & contact-service
- 📂 Same specs-folder approach as Scenario 1

Scenario 2 — Approach

📌 Person Service

- Specs folder with example
- example.zip = target service
- Ideal reference material

📌 Contact Service

- Simpler service to convert
- Less reference material provided
- Higher risk of model drift

Scenario 2 — Result

✅ Person Service

- Conversion worked well
- Model had the target as example.zip
- No surprise — ideal setup

⚠️ Contact Service

- Code conversion worked
- Tests failed repeatedly
- Paused after 60 mins without success

💡 Takeaway — Human in the Loop

Model excels at REST controller & persistence conversion

→ Final tests and edge cases still need manual human-in-the-loop fixes

👀 The more reference material you provide, the better the result